

منصة التبرع بالدم

Blood Donation Platform

وثيقة المشروع الشاملة

مشروع منصة رقمية تربط محتاجي الدم بالمتبرعين القريبين في الوقت الفعلي باستخدام ASP.NET Core و Angular و

.Multi-Agent AI Pipeline

البند	التفاصيل
نوع المشروع	مشروع تخرج — Full Stack Web Application
الفكرة	منصة تربط المرضى أو المستشفيات بمتبرعين مطابقين لفصيلة الدم وقريبين جغرافياً.
Stack	ASP.NET Core 8 + Angular 17 + SQL Server
النمط المعماري	Clean Architecture + CQRS + AI Agent
الفريق	5 أعضاء
المدة الزمنية	5 شهور

القسم الأول: فكرة المشروع والجانب التجاري

Business Overview

١.١ المشكلة التي يحلها المشروع

عند احتياج مريض إلى دم بشكل طارئ، تبدأ رحلة بحث غير منظمة تعتمد غالباً على الاتصالات الشخصية ومنشورات السوشيال ميديا. هذه العملية قد تستغرق ساعات، وفي حالات الطوارئ كل دقيقة قد تكون حاسمة. المشكلة الأساسية هي عدم وجود نظام مركزي ذكي يربط المحتاجين بالمتبرعين المناسبين بسرعة وموثوقية.

١.٢ الحل المقترح

المنصة تقترح حلاً رقمياً يربط طالب الدم بأقرب المتبرعين المطابقين خلال دقائق، مع استخدام Multi-Agent AI Pipeline لتحليل درجة الإلحاح، توسيع نطاق البحث تلقائياً عند الحاجة، إرسال إشعارات فورية، وتحديث حالة الطلب لحظة بلحظة.

Complete Flow — ١.٣ رحلة المستخدم

#	الخطوة	الإجراء	التقنية المستخدمة
1	رفع طلب دم	المريض يحدد الفصيلة والمستشفى والموقع.	Angular Form + REST API
2	تحليل الطلب	AI Agent يحدد مستوى الإلحاح ويجهز رسالة مناسبة.	OpenAI API + .NET Service
3	البحث الجغرافي	النظام يبحث عن متبرعين في نطاق 15 كم.	EF Core + Haversine Formula
4	توسيع البحث	لو مفيش نتائج، يتم التوسيع إلى 30 كم أو فواصل متوافقة.	AI Agent Logic
5	إشعار المتبرعين	إرسال إشعار فوري للمتبرعين المناسبين.	SignalR Hub
6	قبول الطلب	المتبرع يقبل الطلب من التطبيق.	Angular + PUT API
7	تحديث الحالة	المريض والمستشفى يتلقون تحديثاً فورياً.	SignalR + Dashboard
8	إكمال التبرع	المستشفى يؤكد إتمام عملية التبرع.	Hospital Dashboard

١.٤ أصحاب المصلحة

الدور	من هو؟	ماذا يفعل داخل المنصة؟
Patient	أي شخص يحتاج إلى دم	يرفع طلب طارئ ويتابع حالته لحظة بلحظة.
Donor	متبرع مسجل بفصيلة دمه	يرى الطلبات القريبة ويقبل المناسب

		منها.
Hospital	مستشفى أو مركز طبي	يراقب الطلبات ويؤكد إتمام التبرع.
Admin	مدير النظام	يدير المستخدمين والطلبات ويشاهد الإحصائيات.

١.٥ القيمة التجارية والاجتماعية

المعيار	قبل المنصة	بعد المنصة
وقت إيجاد متبرع	من ساعتين إلى أربع ساعات	أقل من خمس دقائق في الحالات المناسبة
نطاق البحث	معارف شخصية فقط	كل المتبرعين في النطاق الجغرافي
مطابقة الفصيلة	يدوية وغير مضمونة	مطابقة تلقائية مع فصائل متوافقة
متابعة الطلب	لا توجد تحديثات لحظية	تحديثات فورية عبر Dashboard
سجل التبرعات	ورقي أو مفقود	رقمي وقابل للبحث والتحليل

القسم الثاني: المعمارية التقنية

Technical Architecture

٢.١ نظرة عامة على Clean Architecture

المشروع مبني على Clean Architecture بحيث تكون طبقة ال Domain هي نواة النظام، ولا تعتمد على أي طبقة خارجية. الاعتمادية دائماً تتجه للداخل، مما يجعل الكود أسهل في الاختبار والصيانة والتطوير.

الطبقة	المحتوى	المسؤولية
Domain Layer	Entities, Enums, Repository Interfaces	تمثل منطق المشروع الأساسي بدون الاعتماد على أي Library خارجية.
Application Layer	CQRS Commands/Queries, DTOs, Service Interfaces	تنفيذ حالات الاستخدام وفصل القراءة عن الكتابة.
Infrastructure Layer	EF Core, SQL Server, Repositories, Services, SignalR, AI Agent	التعامل مع قاعدة البيانات والخدمات الخارجية والبنية التحتية.

API Layer	Controllers, Middleware, Swagger, JWT, CORS	نقطة دخول النظام واستقبال طلبات .Frontend
-----------	---	---

٢.٢ نمط CQRS مع MediatR

CQRS يعني فصل عمليات القراءة Queries عن عمليات الكتابة Commands. كل عملية لها Class مستقل و Handler خاص بها، مما يقلل التداخل بين الوظائف ويسهل الاختبار.

النوع	Class	Handler	الوظيفة
Command	RegisterCommand	RegisterHandler	إنشاء حساب مستخدم جديد
Command	LoginCommand	LoginHandler	تسجيل الدخول وإرجاع JWT
Command	CreateBloodRequestCommand	CreateBloodRequestHandler	إنشاء طلب دم وتفعيل AI Agent
Command	AcceptBloodRequestCommand	AcceptBloodRequestHandler	قبول الطلب بواسطة متبرع
Command	CompleteBloodRequestCommand	CompleteBloodRequestHandler	تأكيد إتمام التبرع
Query	GetNearbyRequestsQuery	GetNearbyRequestsHandler	إرجاع الطلبات القريبة من المتبرع
Query	GetMyRequestsQuery	GetMyRequestsHandler	إرجاع طلبات المريض الخاصة
Query	GetDashboardStatsQuery	GetDashboardStatsHandler	إحصائيات لوحة التحكم

القسم الثالث: تصميم قاعدة البيانات

Database Design — ERD

٣.١ جدول Users

الوصف	القيود	النوع	الحقل
المعرف الفريد للمستخدم	Primary Key	Guid	Id

FullName	string(100)	Required	الاسم الكامل
Email	string(200)	Unique, Required	البريد الإلكتروني
Phone	string(20)	Optional	رقم الهاتف
PasswordHash	string	Required	كلمة المرور مشفرة باستخدام BCrypt
BloodType	enum/string	Required	فصيلة الدم
Role	enum/string	Required	Patient / Donor / Hospital / Admin
Latitude	double	Default 0	خط العرض لموقع المستخدم
Longitude	double	Default 0	خط الطول لموقع المستخدم
IsAvailable	bool	Default true	هل المتبرع متاح حالياً؟
LastDonationDate	?DateTime	Nullable	آخر تاريخ تبرع
CreatedAt	DateTime	Auto	تاريخ إنشاء الحساب

٣.٢ جدول BloodRequests

الوصف	القيود	النوع	الحقل
المعرف الفريد للطلب	Primary Key	Guid	Id
الفصيلة المطلوبة	Required	enum/string	BloodType
Normal / Urgent / Critical	Required	enum/string	Urgency
Pending / Accepted / Completed / Cancelled	Default Pending	enum/string	Status
خط عرض المستشفى	Required	double	Latitude
خط طول المستشفى	Required	double	Longitude
اسم المستشفى	Required	string(200)	HospitalName
ملاحظات إضافية	Optional	string(500)	Notes
وقت إنشاء الطلب	Auto	DateTime	CreatedAt
وقت قبول الطلب	Nullable	?DateTime	RespondedAt
المريض صاحب الطلب	Foreign Key	Guid	PatientId

DonorId	?Guid	Nullable FK	المتبرع الذي قبل الطلب
---------	-------	-------------	------------------------

٣.٣ العلاقات بين الجداول

العلاقة	الوصف	Delete Behavior
User(Patient) → BloodRequests	مريض واحد يمكنه إنشاء عدة طلبات دم.	Restrict
User(Donor) → AcceptedRequests	متبرع واحد يمكنه قبول عدة طلبات على فترات مختلفة.	Restrict
BloodRequest.PatientId	كل طلب مرتبط بمريض واحد.	Required
BloodRequest.DonorId	الطلب يرتبط بمتبرع عند القبول فقط.	Optional / Nullable

Angular 17 — القسم الرابع: الواجهة الأمامية

Frontend — Angular

٤.١ لماذا Angular وليس React؟

المعيار	Angular	React
النوع	Framework كامل	Library للـ UI فقط
اللغة الأساسية	TypeScript بشكل أساسي	JavaScript أو TypeScript
إدارة الحالة	RxJS و Services مدمجة	غالباً يحتاج Redux/Zustand
Forms	Reactive Forms مدمجة	يحتاج مكتبات إضافية
HTTP Client	مدمج	غالباً Axios أو Fetch
Enterprise Apps	مناسب جداً للأنظمة الكبيرة	مرن لكنه يحتاج قرارات معمارية أكثر

٤.٢ هيكل مشروع Angular

- core: يحتوي على models و services و guards و interceptors.

• shared: يحتوي على مكونات مشتركة مثل navbar و loading و blood-type-badge.

• features/auth: صفحات login و register.

• features/patient: إنشاء طلب ومتابعة طلبات المريض.

• features/donor: الطلبات القريبة والخريطة.

• features/hospital: إدارة الطلبات وتأكيدها.

• features/admin: Dashboard والإحصائيات.

٤.٣ الصفحات والمكونات الأساسية

الوظيفة	من يراها؟	Route	الصفحة
تسجيل الدخول وإرجاع JWT	الجميع	login/	LoginComponent
إنشاء حساب جديد	الجميع	register/	RegisterComponent
إحصائيات ورسوم بيانية	Admin/Hospital	dashboard/	DashboardComponent
رفع طلب دم جديد	Patient	request-create/	CreateRequestComponent
متابعة طلبات المريض	Patient	my-requests/	MyRequestsComponent
عرض الطلبات القريبة	Donor	nearby/	NearbyRequestsComponent
إدارة الطلبات	Hospital	manage/	RequestsManagementComponent
تحديث البيانات والتوفر	Donor	profile/	ProfileComponent

القسم الخامس: ال Multi-Agent AI Pipeline

Multi-Agent AI Pipeline

الجزء الذكي في المشروع لم يعد مجرد AI Agent واحد يقوم بكل شيء، بل تم تصميمه كـ Multi-Agent Pipeline منظم. كل Agent له مسؤولية محددة داخل رحلة طلب الدم: تحديد الأولوية، التحقق الطبي، المطابقة الجغرافية، ثم إرسال الإشعارات الفورية.

هذا التصميم يجعل النظام أقرب للأنظمة الواقعية؛ لأنه يقسم القرار إلى مراحل واضحة، سهلة الاختبار والصيانة، ويمكن تطوير كل مرحلة لاحقًا دون التأثير على باقي النظام.

5.1 الشكل العام لل Pipeline

New Blood Request → Triage Agent → Medical Validator Agent → Geo-Matching Agent → Notification Agent → Donor Response

كل مرحلة تستقبل ناتج المرحلة السابقة وتضيف عليه قرارًا جديدًا، إلى أن يصل النظام لقائمة المتبرعين الأنسب ويرسل لهم إشعارات لحظية.

5.2 مراحل عمل ال Multi-Agent Pipeline

المرحلة	ال Agent	المسؤولية	الناتج المتوقع
1	Triage Agent	تحليل طلب الدم وتحديد درجة الأولوية حسب الحالة والملاحظات	Priority Level
2	Medical Validator Agent	فلتر المتبرعين حسب توافق فصائل الدم، التوفر، وآخر تاريخ تبرع	Medically Valid Donors
3	Geo-Matching Agent	حساب المسافة وترتيب المتبرعين من الأقرب إلى الأبعد	Sorted Donors By Distance
4	Notification Agent	إرسال إشعارات فورية ومناسبة للمتبرعين المختارين	Real-Time Notifications

5.3 Triage Agent — تحديد الأولوية

يقوم Triage Agent بتحليل بيانات طلب الدم فور إنشائه لتحديد درجة الإلحاح المناسبة: Urgent أو Normal أو Critical. هذه المرحلة مهمة لأن الطلبات الحرجة يجب أن تحصل على أولوية أعلى في البحث والإشعارات.

- يعتمد على فصيلة الدم المطلوبة، مستوى الحالة، ملاحظات الطلب، اسم المستشفى، ووقت إنشاء الطلب.
- الناتج الأساسي: Priority Level يتم تمريره لباقي مراحل ال Pipeline.

5.4 Medical Validator Agent — التحقق الطبي من المتبرعين

بعد تحديد الأولوية، يقوم Medical Validator Agent باستبعاد أي متبرع غير مناسب طبياً أو غير متاح، حتى لا تصل إشعارات عشوائية لأشخاص لا يمكنهم التبرع فعليًا.

- يتحقق من توافق فصيلة دم المتبرع مع فصيلة الدم المطلوبة.
- يتحقق من IsAvailable للتأكد أن المتبرع متاح حاليًا.
- يراعي LastDonationDate لتجنب ترشيح متبرع تبرع مؤخرًا.

• مثال: طلب +A يمكن أن يقبل متبرعين من +A و -A و +O و -O.

5.5 Geo-Matching Agent — المطابقة الجغرافية

يقوم Geo-Matching Agent بحساب المسافة بين موقع المستشفى وموقع كل متبرع باستخدام Haversine Formula، ثم يرتب المتبرعين من الأقرب إلى الأبعد لضمان سرعة الاستجابة.

• يبدأ البحث داخل نطاق قريب مثل 15 km.

• إذا لم يجد عددًا كافيًا من المتبرعين، يمكن توسيع النطاق تلقائيًا إلى 30 km.

• الناتج الأساسي: قائمة متبرعين مرتبة حسب المسافة والأولوية.

5.6 Notification Agent — إرسال الإشعارات

يقوم Notification Agent باستقبال قائمة المتبرعين المناسبة ويرسل لهم إشعارات فورية باستخدام SignalR، بدون الحاجة لعمل Refresh للصفحة.

• الإشعار يحتوي على فصيلة الدم المطلوبة، درجة الإلحاح، اسم المستشفى، المسافة التقريبية، وزر قبول الطلب.

• في الحالات الحرجة يمكن استخدام صياغة أكثر وضوحًا واستعجالًا داخل رسالة الإشعار.

• بمجرد قبول متبرع للطلب، يتم تحديث حالة الطلب في Dashboard الخاصة بالمريض أو المستشفى لحظيًا.

5.7 ملاحظات مهمة على التصميم

الملاحظة	التوضيح
Multi-Agent وليس Agent واحد	كل Agent مسؤول عن خطوة محددة داخل القرار، مما يقلل التعقيد ويزيد قابلية الاختيار.
ليس كل جزء يحتاج GPT	حساب المسافة وتوافق فصائل الدم يتمان بقواعد وخوارزميات واضحة، وليس LLM.
استخدام الذكاء هنا Decision Pipeline	الذكاء في ترتيب القرار وتشغيل المراحل تلقائيًا وليس فقط توليد نص.
قابلية التطوير	يمكن إضافة Agents لاحقًا مثل Fraud Detection أو Donor Fatigue بدون إعادة بناء النظام.
قابلية الشرح	يمكن تتبع سبب اختيار كل متبرع: متوافق طبيًا، متاح، قريب، وتم إشعاره.

5.8 لماذا هذا التصميم أفضل؟

التصميم متعدد الوكلاء أفضل من وضع كل المنطق داخل Service واحد ضخم؛ لأنه يفصل المسؤوليات بوضوح. يمكن اختبار Triage Agent وحده، و Medical Validator وحده، و Geo-Matching وحده، و Notification Agent وحده. هذا يجعل المشروع أقوى تقنيًا وأسهل في العرض والمناقشة.

5.9 Agents يمكن إضافتها مستقبلاً

Agent	الفكرة المستقبلية
Fraud Detection Agent	اكتشاف الطلبات الوهمية أو المتكررة بشكل غير طبيعي.
Donor Fatigue Agent	منع إرسال إشعارات كثيرة لنفس المتبرع في فترة قصيرة.
Hospital Load Agent	ترتيب الطلبات حسب ضغط المستشفيات والحالات الحرجة.
Analytics Agent	تحليل معدلات الاستجابة، أوقات القبول، ونسبة اكتمال التبرعات.

القسم السادس: ال API Endpoints

API Reference

Method	Endpoint	Role	الوظيفة
POST	api/auth/register/	Public	تسجيل مستخدم جديد
POST	api/auth/login/	Public	تسجيل الدخول وإرجاع JWT
POST	api/requests/	Patient	رفع طلب دم جديد
GET	api/requests/nearby?/ =lat=&lng=&r	Donor	إرجاع طلبات قريبة من موقع المتبرع
GET	api/requests/my/	Patient	طلبات المريض الخاصة
GET	api/requests/all/	Admin/Hospital	كل الطلبات
PUT	api/requests/{id}/accept/	Donor	قبول طلب التبرع
PUT	api/requests/{id}/complete/	Hospital/Admin	تأكيد إتمام التبرع
PUT	api/donors/availability/	Donor	تحديث حالة التوفر
PUT	api/donors/location/	Donor	تحديث موقع GPS
GET	api/dashboard/stats/	Admin/Hospital	إحصائيات عامة
WS	hubs/notifications/	All JWT	إشعارات فورية باستخدام SignalR

القسم السابع: التقنيات المستخدمة بالكامل

Full Technology Stack

الطبقة	التقنية	الإصدار	الغرض
Backend	ASP.NET Core	8.0	Web API الرئيسي
ORM	Entity Framework Core	8.0	الوصول لقاعدة البيانات و Migrations
Database	SQL Server / LocalDB	Latest	حفظ البيانات
Messaging	MediatR	x.12	تطبيق CQRS
Real-Time	SignalR	8.0	الإشعارات الفورية

Auth	JWT Bearer	8.0	المصادقة عبر Token
Security	BCrypt.Net	x.4	تشفير كلمات المرور
AI	OpenAI API	GPT model	تحليل الطلبات وتوليد رسائل ذكية
Docs	Swagger / OpenAPI	x.6	توثيق API تلقائياً
Frontend	Angular	17	إطار الواجهة الأمامية
Language	TypeScript	x.5	لغة Frontend
Charts	Chart.js / ngx-charts	Latest	رسوم Dashboard
Maps	Leaflet + ngx-leaflet	Latest	خريطة تفاعلية للمتبرعين

القسم الثامن: هيكل الملفات الكامل

Complete File Structure

Backend — .NET Solution ٨.١

- BloodDonation.Domain: يحتوي على Entities و Enums و Interfaces.
- BloodDonation.Application: يحتوي على DTOs و CQRS Features و Service Interfaces.
- BloodDonation.Infrastructure: يحتوي على AppDbContext و Repositories و Services و SignalR Hub.
- Multi-Agent AI Pipeline
- BloodDonation.API: يحتوي على Controllers و Program.cs و appsettings.json.

Frontend — Angular ٨.٢

- .src/app/core: services, guards, interceptors, models
- .src/app/shared: reusable components and pipes
- .src/app/features/auth: login and register
- .src/app/features/patient: create request and my requests
- .src/app/features/donor: nearby requests and map view

- .src/app/features/hospital: requests management
- .src/app/features/admin: dashboard and statistics

القسم التاسع: خطة التنفيذ والفريق

Implementation Plan & Team

٩.١ خطة التنفيذ خلال 5 شهور

الشهر	التركيز	المهام الأساسية
الأول	التأسيس والإعداد	إعداد البيئة، إنشاء Clean Architecture، بناء Domain، إعداد EF Core، تنفيذ Register/Login، إنشاء مشروع Angular.
الثاني	الميزات الأساسية	بناء CQRS لطلبات الدم، API، Haversine، Endpoints، صفحات Patient و Donor، تفعيل Role-based Authorization.
الثالث	Real-Time والخريطة	إعداد SignalR، إشعارات المتبرعين، دمج Leaflet Map، وتحديث موقع المتبرع.
الرابع	Dashboard و AI Agent	بناء BloodDonationAgent، منطق توسيع البحث، خوارزمية الفصائل المتوافقة، Dashboard Charts، وتأكيـد التبرع.
الخامس	الاختبار والنشر	Unit/Integration Tests، تحسين UI، Responsive Design، نشر Backend و Frontend، تجهيز README والعرض.

٩.٢ توزيع الأدوار على الفريق

المسؤوليات	الدور الأساسي	العضو
Domain Layer, Application Layer, CQRS Commands & Queries.	Backend Lead	العضو 1

العضو 2	Backend / DB	Infrastructure, EF Core, Repositories, Database Migrations
العضو 3	Frontend Lead	Angular Pages, UI Design, Routing, Guards, Interceptors
العضو 4	Integration	AI Agent, SignalR, Real-Time, Maps وربط Frontend مع Backend
العضو 5	DevOps / QA	Testing, Deploy, README, Documentation, وتجهيز عرض التخرج.

القسم العاشر: خطوات التشغيل والإعداد

Setup & Run Guide

١٠.١ المتطلبات الأساسية

البرنامح	الإصدار	الغرض
NET 8 SDK.	x.8	تشغيل Backend
Visual Studio 2022	Community	تطوير وتشغيل Solution
SQL Server / LocalDB	Latest	قاعدة البيانات
Node.js	LTS +18	تشغيل Angular
Angular CLI	+17	إنشاء وتشغيل Frontend
Git	Latest	إدارة الكود

١٠.٢ تشغيل Backend

#	الوصف	الأمر / الإجراء
1	فتح الـ Solution داخل Visual Studio.	افتح BloodDonation.sln
2	تحديث Connection String و JWT و OpenAI Key.	عدّل appsettings.json
3	إنشاء أول Migration.	dotnet ef migrations add InitialCreate
4	إنشاء الجداول داخل SQL Server.	dotnet ef database update

5	F5 أو dotnet run	تشغيل API على localhost.
6	افتح swagger/	اختبار الـ API من Swagger UI.

١٠.٣ تشغيل Frontend

#	الأمر	الوصف
1	cd frontend	الدخول إلى مجلد Frontend.
2	npm install	تحميل الحزم المطلوبة.
3	ng serve	تشغيل Angular Dev Server.
4	http://localhost:4200	فتح واجهة المستخدم في المتصفح.
5	ng build --prod	بناء نسخة الإنتاج عند الحاجة.

ملاحظات تشغيل: Backend غالباً يعمل على http://localhost:5000، و Swagger على http://localhost:5000/swagger. أما Frontend فيعمل على http://localhost:4200. لتفعيل Multi-Agent AI Pipeline يجب إضافة OpenAI ApiKey داخل appsettings.json.

تم إعداد هذه الوثيقة بصياغة منظمة وقابلة للتعديل داخل Word.